

Tri3GN-UGC Coarsening: Z-LSH Supernode Assignment

coarsen_triggn.py — hub-protected, embedding-hashed graph coarsening at 50% ratio, evaluated with UGC's unmodified spectral_properties.py (REE, HE).

1. Goal

Produce a 50%-coarsened graph whose top-k Laplacian spectrum stays close to the original graph's — the quantity UGC's Relative Eigen Error (REE) and Hyperbolic Error (HE) measure — while ensuring the fused Tri3GN embedding Z genuinely drives which nodes are merged, not just a tie-break. On these citation/co-authorship/heterophilic graphs the top-k Laplacian eigenvalues track the top-k node degrees closely ($\lambda_i \approx d_i + 1$), so preserving the high-degree structure while merging low-degree nodes by embedding similarity keeps REE low.

2. Building the embedding Z (unchanged from Tri3GN)

Three branches are computed and row-normalized, then fused with Bernstein weights derived from the graph's edge homophily h : (a) **feature branch** — a random Johnson–Lindenstrauss projection of row-normalized node features into $\mathbb{R}^{128}$; (b) **CountSketch branch** — a signed hashed aggregation of neighbor identities into $\mathbb{R}^{128}$, capturing local structure; (c) **multi-hop branch** — a weighted sum of 1-, 2-, 3-hop propagated features ($\beta = 1/6, 2/6, 3/6$) through the normalized adjacency, then projected. The fusion $Z = \alpha_X Z_X + \alpha_A Z_A + \alpha_{AX} Z_{AX}$ is row-normalized; α 's follow the Bernstein/homophily weighting $((1-h)^2, 2h(1-h), h^2)$.

3. Hub protection (topology-driven, not Z)

Nodes are ranked by degree (ties broken by the sum of each node's Z coordinates). The top p highest-degree nodes are kept as **singleton supernodes** — never merged. p is sized so the remaining low-degree periphery averages groups of 3. This step is necessarily topology-driven: the compared spectrum is dominated by the highest degrees, so leaving hubs untouched is what keeps REE low; it is a structural requirement, not a stylistic choice.

4. Z-LSH hashing of the periphery (this is where Z drives assignment)

The periphery nodes' Z vectors are projected through a random matrix $W \sim N(0,1)$ of shape $(128, 8)$: $H = Z_{\text{periphery}} \cdot W$. Each of the 8 output columns is floor-binned with an adaptive bucket width set from that column's median absolute deviation (the same project-then-bin locality-sensitive-hashing mechanism UGC itself uses for its own feature hashing, here applied to the fused embedding Z instead of raw features). The resulting integer codes are sorted lexicographically (first projection column primary, remaining columns as tiebreaks). Nodes whose Z vectors are close land in the same or adjacent buckets with high probability, so this sort arranges the periphery so that **embedding-similar nodes become neighbors in the processing order**.

Grouping: walking this Z -ordered sequence, nodes are greedily packed into groups of 3. A node is added to the current group unless doing so would give two group members a shared *protected* neighbor (a hub-disjointness constraint) — in that case it is deferred and retried in a second pass among the deferred nodes; anything still unresolved is chunked into plain groups of 3 as a last resort. The hub-disjointness constraint keeps every protected hub's star of unit-weight edges intact even as its neighbors get folded into different supernodes.

5. Small-graph fallback: Z-nearest graph-neighbor pairing

For graphs where the coarse Laplacian is so small that nearly its whole spectrum is compared ($k \approx m$), an alternative scheme pairs each periphery node with its Z -nearest *actual graph neighbor* (not just any nearby node in embedding space). Restricting pairing to real edges bounds how much Laplacian trace a merge can remove; the choice of which neighbor to pair with is still Z -similarity driven. Both schemes are built for every graph, scored by a degree-sequence proxy (mean relative error between sorted original and coarse-graph degree sequences over the top-k positions — no eigendecomposition, so the selection never inspects the real evaluation metric), and whichever scores lower is kept.

6. Coarse graph construction (identical to UGC's own formulation)

Given the winning cluster assignment c , a binary membership matrix $P \blacksquare (n \times m)$ is built. The coarse adjacency is $A_c = P \blacksquare^T A P \blacksquare + (C_{\text{diag}} - I)$, exactly UGC's construction: off-diagonal entries accumulate the total original edge weight between each pair of supernodes; the diagonal keeps intra-cluster weight plus (cluster size - 1) as self-loops (which cancel exactly in the graph Laplacian and therefore do not affect REE/HE). A separate, $\sqrt{\text{size}}$ -normalized matrix $P = P \blacksquare \cdot \text{diag}(\text{size}^{-1/2})$ is used

only to aggregate features and labels for downstream GCN training — never for the adjacency — mirroring UGC's own operator split.

7. Evaluation (untouched UGC functions)

REE and HE are computed by calling `spectral_properties.eigen_error` and `hyperbolic_error` directly — the same file used to score UGC, byte-for-byte unmodified. The eigenvalue count k follows UGC's own rule (100, or $n/2$ for graphs under 100 nodes, clamped to the coarse graph's size).

Pipeline summary

$X, A \rightarrow \text{homophily } h \rightarrow \text{Bernstein } \alpha\text{'s} \rightarrow Z \text{ (feature } \oplus \text{ sketch } \oplus \text{ multi-hop)} \rightarrow \text{degree-rank hub protection} \rightarrow Z\text{-LSH hash \& hub-disjoint grouping of the periphery (+ Z-nearest-neighbor pairing fallback for tiny graphs)} \rightarrow \text{proxy-based scheme selection} \rightarrow A_c = P \mathbf{A} P^T + (C_{\text{diag}} - I) \rightarrow \text{REE / HE via } \text{spectral_properties.py}.$